

Examen Práctico

Programación para Sistemas

Julio 2024

INSTRUCCIONES

- Conecte su portátil a la WIFI Eduroam. Tendrá que apuntar la IP en la hoja de firmas que se pasará.
 - Escriba el código pedido en el enunciado en los ficheros solicitados.
 - Compruebe en su entorno local que compila correctamente con las opciones o 'flags' **-Wall -Wextra -Werror -ansi -pedantic** y que la ejecución es correcta.
 - Traslade el código a triqui y vuelva a comprobar la compilación y la ejecución.
 - Entregue su código desde triqui. Para entregar debe ejecutar desde el directorio donde tenga los ficheros solicitados el comando: `entrega.pps julio`
 - Puede entregar varias veces, y la última entrega será la que se corrija. **El tiempo para la realización del examen es de 2h y 15 minutos. Se dispondrá de 15 minutos adicionales para la realización de la entrega.**
 - **Si no ha sido capaz de entregar en triqui, podrá hacer una entrega en Moodle en la tarea correspondiente, pero tendrá una penalización de 2 puntos.**
 - La entrega consistirá de un fichero por apartado : **ej1.c, ej2.c, ej3a.c, ej3b.c, ej3c.c, ej3d.c, y ej4.sh.**
 - Se proporciona en el Moodle de la asignatura un fichero **ex202407.h** con prototipos y declaraciones de tipos, etc. que se comentan en el enunciado, y que **necesariamente** se deberá utilizar incluyéndolo. Dicho fichero no se puede modificar, ni se puede repetir en ficheros entregados lo que ahí aparece.
 - Se incluye así mismo un fichero **ex202407_main.c** con un main con ejemplos de llamadas sencillas a funciones del enunciado por si puede servir de utilidad, y un fichero de entrada `lineas.txt` con los casos de las líneas "ba", "bd" y "aa" que se comentan en el enunciado (las salidas correspondientes al apartado 3.b en `stdout` y `stderr` son `lineas_ej3b_stdout.txt` y `lineas_ej3b_stderr.txt`, respectivamente).
 - Al desarrollar, no se debe incluir en el fichero **ex202407_main.c** código de implementaciones de funciones pedidas. Las funciones pedidas en cada apartado deben estar en los ficheros separados que se indican y que habrá que entregar, y en las pruebas al desarrollar se deben realizar compilaciones de varios ficheros fuente.
 - Los ficheros a entregar, uno por apartado, deben contener sólo lo que se pide en el apartado en cuestión, y puede haber funciones auxiliares (aunque naturalmente no incluir un main).
-

Ejercicio 1. Implemente la función `argumentos_y_entrada` que procese los argumentos recibidos en línea de comandos. El resultado de la función será el descriptor de fichero (puntero `FILE *`) sobre el que se va a leer en otros apartados.

Si al ejecutable no se proporciona un argumento, el programa va a leer de entrada estándar, y el resultado de dicha función será `stdin`. Si se proporciona un argumento, el resultado será el descriptor de fichero asociado a dicho nombre al abrirlo como lectura; si el fichero no existiera o no tuviera permiso de lectura, se devolverá `NULL`. Si se proporciona más de un argumento, el resultado de la función será `NULL`.

[1.5 puntos]

Ejercicio 2. En apartados siguientes se va a almacenar cadenas de caracteres en una tabla, y en este apartado se pide implementar la función `calcular_indice_tabla` que calcula el índice correspondiente a la cadena de caracteres referenciada por el argumento `ele`.

El índice será la suma de los códigos ASCII de los caracteres de una línea (excluyendo el posible carácter nueva línea) módulo N , siendo N el tamaño de la tabla (número de índices de la tabla).

Por ejemplo, para el caso de una línea que sea "ba", si N fuera 3, el cálculo del índice que debe realizar `calcular_indice_tabla` es: $(98+97) \% 3 = 0$, donde 98 y 97 son los códigos ASCII de 'b' y 'a', respectivamente.

Se puede suponer que `ele` no es NULL y que apunta a una cadena de caracteres con al menos un carácter.

Nota: se recuerda que, en lenguaje C, un carácter de tipo `char` es, en muchos aspectos, intercambiable con un entero con su código ASCII. Por ejemplo, si `car` y `codigo_ascii` son dos variables, respectivamente, `char` e `int`, se puede hacer una asignación `codigo_ascii = car` y la variable `codigo_ascii` contendría el código ASCII del carácter en `car`.

[1.5 puntos]

NOTA: si no ha realizado el apartado 2 correctamente como se pide, a efectos de continuar la realización del resto de los ejercicios y de realizar algunas pruebas, puede utilizar la siguiente implementación:

```
int calcular_indice_tabla(char *ele, int N) {
    return *ele % N;
}
```

En los siguientes apartados, se va a emplear una tabla indexada muy simple (y limitada) que sólo va a poder guardar una línea por cada índice de la tabla (el cual se va a calcular por la función `calcular_indice_tabla`). Sea un tamaño de la tabla igual a 3, como ejemplo de tabla muy reducida. Una tabla de tamaño 3, tras su creación tendría la siguiente estructura (ver en `ex202407.h` la declaración de la estructura asociada):

```
struct tabla_s -----
    | 3 (tamanno) |
    -----
    |(array_tabla)|-----> -----
    -----
                                |NULL|
                                -----
                                |NULL|
                                -----
                                |NULL|
                                -----
```

La estructura `struct tabla_s` tiene un campo `tamanno` que contiene el tamaño (un entero, el cual en el caso de la figura es 3), y un campo `array_tabla` que es un puntero que apunta a un array de memoria dinámica (en este ejemplo, de tamaño 3).

Si, por ejemplo, la primera línea de la entrada fuera "ba", dicha línea tendría un índice asociado igual a 0 (ver ejemplo de cálculo comentado en el apartado 2). Tras la operación de inserción la tabla quedaría como :

```
struct tabla_s -----
    | 3 (tamanno) |
    -----
    |(array_tabla)|-----> -----
    -----
                                |  |-----> "ba"
                                -----
                                |NULL|
                                -----
                                |NULL|
                                -----
```

La primera posición con índice 0 del array apuntará a la línea “ba” (más exactamente, a una copia de la misma que excluye el posible carácter de nueva línea que pudiera existir).

En esta sencilla versión de tabla, no se puede guardar más de una línea por índice. Si, por ejemplo, la segunda línea fuera “bd”, el índice asociado sería 0 ($\text{calcular_indice_tabla}$ calcularía: $(98+100) \% 3 = 0$), y no se podría guardar dicha línea en la tabla ya que se ha insertado previamente una línea en dicha posición. Estas líneas que no se puedan guardar se deberán mostrar en salida error.

Si aparece una tercera línea que sea “aa”, su índice correspondiente sería $(97+97) \% 3 = 2$, y tras su inserción en la tabla la situación sería:

```

struct tabla_s -----
      | 3 (tamanno) |
      -----
      |(array_tabla)| -----> -----
      -----
                                |      | ----> "ba"
                                -----
                                |NULL|
                                -----
                                |      | ----> "aa"
                                -----

```

Vemos que se han podido guardar las líneas “ba” y “aa”, mientras que la línea “bd” se mostrará en salida error (no se ha podido almacenar).

Ejercicio 3.a. Implemente la función `crear_tabla_vacia` que crea una tabla `struct tabla_s` de tamaño N vacía y devuelva un puntero apuntando a la misma.

Si hubiera algún error relacionado con la asignación de memoria dinámica, se deberá terminar la ejecución, devolviendo al sistema operativo el número status 71.

Se puede ver en la primera figura de `struct tabla_s` un diagrama de una tabla vacía de tamaño N igual 3.

[1 punto]

Ejercicio 3.b. Implemente la función `mostrar_e_insertar_lineas` tal que:

- lea las líneas de la entrada designada por `fp`,
- muestre todas las líneas leídas en salida estándar (similarmente a como lo haría el comando `cat` de Unix), y
- en el caso de que las líneas no sean vacías (que no contengan sólo un salto de línea), debe intentar insertarlas (más bien, copias de las líneas) en la estructura apuntada por `ptabla` excluyendo el posible carácter nueva línea que pueda tener. Como se ha comentado anteriormente, es posible que no se pueda insertar una línea en la tabla si su índice (calculado según `calcular_indice_tabla`) ya está ocupado. En el caso de que el índice esté ocupado, se debe escribir en salida error la línea que no se ha podido insertar.

Las líneas tendrán un tamaño máximo de 80 caracteres (incluyendo el posible salto de línea).

Se puede suponer que `fp` es un descriptor de fichero de entrada ya correctamente abierto (como el que devolvería la función `argumentos_y_entrada` en caso de situación de no error).

Si hubiera algún error relacionado con la asignación de memoria dinámica, se deberá terminar la ejecución, devolviendo al sistema operativo el número status 71.

[2 puntos]

Ejercicio 3.c. Implemente la función `mostrar_tabla` que muestre en salida estándar todas las líneas introducidas en la tabla apuntada por `ptabla`, recorriendo los índices de `array_tabla` empezando por el índice 0.

[1 punto]

Ejercicio 3.d. Implemente la función `reiniciar_y_liberar_array_de_tabla` de reinicio y de liberación de memoria que deberá (1) poner a 0 el campo `tamanno` de la estructura apuntada por `ptabla`, (2) liberar toda la memoria dinámica, y (3) establecer que el campo `array_tabla` sea `NULL`.

[1 punto]

Ejercicio 4. Implemente un script en Bash que acepte 5 argumentos: el primero sería un ejecutable que se pueda llamar, el segundo argumento un fichero de entrada, el tercero un directorio, y los dos siguientes son nombres de ficheros que deberán almacenar las salidas estándar y error, respectivamente, de la ejecución del ejecutable.

- Si hay error en el número de argumentos se deberá parar la ejecución devolviendo al sistema operativo el número status 1.
- Si el primer argumento no es un ejecutable, se parará la ejecución con número status 2.
- Si el segundo argumento no es un fichero que exista y que se pueda leer, se parará la ejecución con número status 3.
- El tercer argumento será el nombre de un directorio donde se van a guardar las salidas estándar y error. Si el directorio no existe, se deberá crear y, en el caso de no tener éxito, se deberá parar la ejecución con número status 4. Nota: se puede suponer que el directorio a crear es simple (no hay varios niveles).
- El script deberá lanzar el ejecutable sobre el fichero de entrada y, en el directorio reseñado en el tercer argumento, guardar las salidas estándar y error en los ficheros indicados en los argumentos cuarto y quinto, respectivamente.

[2 puntos]
